Master Roller Shutter Control V89

Home Assistant blueprint – sun-position shading, extreme-heat protection, ventilation and walk-through logic

Complete documentation · As of October 2026 · Version V89 · Blueprint domain: automation

Contents

1. What the blueprint does
2. How it works: the priority chain
3. Prerequisites and required entities
4. Creating helpers and template sensors
5. Window sensors: tilted vs. fully open
6. Installing the blueprint
7. Complete input reference
8. Recommended settings by facade orientation
9. The manual override
10. Working with a fire-alarm automation (including acknowledged reset and dashboard card)
11. Troubleshooting and debugging
12. Known limitations
13. Changelog

1. What the blueprint does

The blueprint controls a single roller shutter fully automatically throughout the day. One automation is created from the blueprint per shutter, so every room can have its own facade orientation, temperature thresholds and positions.

It combines six jobs that would otherwise need separate automations:

- **Morning opening** – based on time of day, sunrise, presence, weekday, public holiday and house mode.
- **Sun-position shading** – the shutter only lowers when the sun actually hits this facade (azimuth window per orientation), when it is bright enough (PV power or lux) and when the temperature justifies it.
- **Extreme-heat protection** – at very high indoor and outdoor temperatures the shutter moves to a separate, much lower position. With hysteresis to prevent oscillation.
- **Ventilation position** – a tilted window keeps the shutter at a defined minimum height.
- **Walk-through and cross-ventilation logic** – a fully opened window or patio door raises the shutter so nobody gets locked out and no manual intervention is needed.
- **Evening closing** – after sunset with a configurable offset, taking open windows into account.

> **Core idea:** the blueprint never decides on a single condition but follows a fixed priority chain. Safety beats comfort, comfort beats energy.

2. How it works: the priority chain

On every trigger the blueprint walks a `choose` chain from top to bottom. The first matching branch wins; all others are skipped.

Prio	Branch	Condition and effect
0	Fire alarm	If the fire-alarm boolean is `on`, execution ends immediately with `stop`. The blueprint moves nothing – it stays quiet so it does not close shutters that an external fire-alarm automation has opened as an escape route. See chapter 10.
1a	Walk-through / cross-ventilation	A sensor from *window fully open* reports `on`. The shutter raises: • no shading active → to `max_oeffnung` (fully open) • shading or extreme-heat protection active → only to `durchgangs_pos` Also applies at night, before the morning opening time and from a manually closed state. The shutter is never lowered here, only raised.
1b	Ventilation (tilt)	A sensor from *window ventilation* reports `on` and no fully-open sensor is active. The shutter moves to at least `lueftungs_pos` so the curtain does not rest on the tilted sash.
2	Evening check	At the configured check time (default 22:00) the blueprint verifies whether a window is still fully open and sends a push notification.
3	Closing	Night (sunset plus `sunset_offset`) and no window fully open → the shutter closes. With a tilted window it closes only down to `lueftungs_pos`. Since V87 the offset lives in the night condition itself and therefore applies to every trigger, not just the sunset one.
4a	Extreme-heat protection	Sun on the facade *and* indoor temperature above the ON threshold *and* outdoor temperature above the ON threshold → the shutter moves to `hitze_extrem_pos`. The mode only ends once indoor or outdoor temperature drops below the respective OFF threshold (hysteresis).
4b	Normal shading	Sun on the facade *and* heat alarm *and* enough light (PV or lux) → the shutter moves to the shading position. Configurable separately for morning and afternoon.
4c	Raising	No sun protection needed (sun gone, cloudy, cool) → the shutter moves to `max_oeffnung`. **Below the winter**

lock this branch works reliably since V89 – before, it stayed blocked there.

When does the automation actually run?

Triggers are: state changes of the window sensors, presence detectors, PV power (debounced by 2 min), sunset, the *latest opening* and *evening check* times, a change of house mode – plus a `time_pattern` every 5 minutes as a safety net. The mode is `restart`: a new trigger aborts a run in progress.

> Since V87 there is no dedicated trigger for the sunset offset. The safety net fires the closing branch as soon as the night condition flips, so the worst-case delay is 5 minutes. A `delay` would be useless here because `mode: restart` aborts any pending run on the next trigger.

The three sun conditions in detail

- `sonne_auf_f` – the sun azimuth falls within the angular range of the chosen orientation and elevation is above 5°. Prevents shading when the sun is not on the window at all.
- `sonne_stark` – PV power above the threshold or lux above the threshold, each sanity-checked against the weather entity. If a sensor fails, a failover based on weather alone takes over.
- `hitze_alarm` – an OR combination: indoor temperature above the indoor threshold, or a rising temperature trend, or outdoor temperature above the outdoor threshold. A winter lock (`raumtemp_min_schwelle`) suppresses everything while the room is too cool. **This lock only works correctly from V89: up to V88, `hitze_alarm` was wrongly always true below the winter lock – see the changelog.**

> **Important to understand:** because `hitze_alarm` is an OR combination, a very low outdoor threshold (e.g. 10 °C) makes the indoor temperature practically irrelevant – above 10 °C outside the condition is always true in summer, and only sun position and brightness decide. That may be intended ("the sun blinds me at 15 °C too"), but it should be a conscious choice.

3. Prerequisites and required entities

What	Required	Description
`cover.*`	yes	Roller shutter with position support (`set_cover_position` and the `current_position` attribute). Pure open/close shutters do not work.
`sun.sun`	yes	Standard integration. Provides azimuth and elevation.
`weather.*`	yes	Any weather entity. Used for plausibility checks and as a failover.
Room temperature sensor	yes	`sensor` with `device_class: temperature`.
Outdoor temperature sensor	yes	Own sensor or e.g. OpenWeatherMap.
`input_select` house mode	yes	Helper with at least the options Zuhause, Abwesend, Urlaub (home, away, holiday).
`input_boolean` fire alarm	yes	Set by the external fire-alarm automation.
Window sensor tilt	no	`binary_sensor`, multiple selection possible.
Window sensor fully open	no	`binary_sensor`, multiple selection possible. Leave empty for windows that are never fully opened.
Presence detector	no	Multiple selection. Triggers the morning "starting signal".
PV power sensor	no	W or kW – the unit is detected and converted automatically.
Illuminance sensor	no	`device_class: illuminance`. One sensor per facade side is ideal.
Trend sensor	no	`binary_sensor`, `on` = room temperature rising. See chapter 4.
Workday sensor	no	For public-holiday detection. See chapter 4.
Notify service	no	Recommendation: a notify group, see chapter 4.

Since V89 the notification-pause timer is no longer needed – the blueprint had stopped evaluating it.

4. Creating helpers and template sensors

4.1 House mode (input_select)

Settings → Devices & Services → Helpers → Create helper → Dropdown. Name: Hausmodus. Write the options exactly like this – the blueprint compares text values:

```
Zuhause
Abwesend
Urlaub
```

Effect: *Urlaub* switches to the late opening time. *Abwesend* and *Urlaub* also allow opening without a presence report.

4.2 Fire-alarm status (input_boolean)

Create helper → Toggle. Name e.g. Brandalarm Status → entity `input_boolean.brandalarm_status`. This boolean is the interface between the smoke-detector automation and the shutter control.

4.3 Workday sensor (public holidays)

Settings → Devices & Services → Add integration → Workday. Select country and region, mark weekends as non-working days. Result: `binary_sensor.workday_sensor`, which is `off` on weekends and holidays.

The blueprint uses it like this: `off` → the late opening time applies (weekend/holiday). If the input is left empty, the blueprint falls back to a plain weekday check – public holidays are then not detected.

4.4 Trend sensor for room temperature

The trend sensor makes shading anticipatory: if the room temperature is rising, shading starts before the threshold is reached. Create it via the Trend integration (Settings → Devices & Services → Add integration → Trend) or in YAML:

```yaml
binary_sensor:
  - platform: trend
    sensors:
      temp_wohnzimmer_rollo:
        friendly_name: "Living room temp rising"
        entity_id: sensor.temperatur_wohnzimmer
        sample_duration: 1800     # 30 minute observation window
        min_gradient: 0.0005      # approx. +0.9 K/h
        device_class: heat
```

Create one such sensor per room and reference it in the corresponding automation.

4.5 Notify group (recommended)

Instead of entering the device service in every automation, use a notify group. When a device changes you then only edit one line. In

`configuration.yaml`:

```
notify:
  - platform: group
    name: handys
    services:
      - service: mobile_app_my_phone
      - service: mobile_app_partner_phone      # optional
```

Afterwards restart Home Assistant (a reload is not enough for notify platforms). In the blueprint you then enter only `handys` as the *phone notification service* – the blueprint builds `notify.handys` from it.

> Notify groups are legacy services and do not create an entity. They therefore do not appear under "States", only under Developer tools → Actions.

5. Window sensors: tilted vs. fully open

The blueprint distinguishes two window states because they have different consequences:

- **Tilted** → the shutter may come down, but only to the ventilation position (otherwise the curtain rests on the sash).
- **Fully open** → the shutter must go up (walk-through, cross-ventilation, no lock-out).

A single reed contact cannot tell these apart – it only sees "sash moving away from frame". There are three practical approaches:

Option A – Two reed contacts (recommended, most robust)

One contact is placed so that it opens on tilt, a second so that it only opens on full opening (typically on the closing side of the sash). Both sensors go directly into the respective blueprint inputs – no templates needed. This is the option with the lowest error potential.

Option B – Handle sensor plus window contact

A window handle sensor (e.g. Aqara, Bosch) reports three states: closed / tilted / open. Combined with an opening contact, both states can be derived via template helpers:

```
template:
  - binary_sensor:
      - name: "Office window tilted"
        unique_id: fenster_buero_kippstellung
        device_class: window
        state: >
          {{ is_state('sensor.fenstergriff_buero', 'tilt') }}

      - name: "Office window fully open"
        unique_id: fenster_buero_ganz_geoeffnet
        device_class: window
        state: >
          {{ is_state('sensor.fenstergriff_buero', 'open') }}
```

The exact state names (`open`, `tilt`, `closed`) depend on the device and integration – check under Developer tools → States.

Option C – Window contact plus separate handle contact

If only an opening contact (`A`) and a contact on the handle (`B`) are available, the logic can be derived like this:

```
template:
  - binary_sensor:
      - name: "Kitchen window tilted"
        unique_id: fenster_kueche_kippstellung
        device_class: window
        state: >
          {{ is_state('binary_sensor.fenster_kueche_contact', 'on')
             and is_state('binary_sensor.griff_kueche_contact', 'on') }}

      - name: "Kitchen window fully open"
        unique_id: fenster_kueche_ganz_geoeffnet
        device_class: window
        state: >
          {{ is_state('binary_sensor.fenster_kueche_contact', 'on')
             and is_state('binary_sensor.griff_kueche_contact', 'off') }}
```

> **Windows without a fully-open sensor:** simply leave the *window fully open* input empty. The walk-through branch then never applies to that shutter – everything else keeps working. Useful for windows that are only ever tilted (guest WC, basement windows).

> **Pitfall:** if the same contact is entered as both the tilt and the fully-open sensor, the fully-open logic permanently blocks the ventilation logic. Only ever assign one of the two inputs per physical sensor.

6. Installing the blueprint

1. Place the YAML file under: `config/blueprints/automation/<folder>/<filename>.yaml`
2. Developer tools → YAML → Reload *automations*. For changes to `configuration.yaml`, do a full restart instead.
3. Settings → Automations & Scenes → Blueprints → select the blueprint → *Create automation*.
4. Fill in all inputs for the first shutter, save, name it.
5. Repeat for every further shutter.

When the blueprint is updated later, it is enough to replace the file and reload the automations. The configured inputs are preserved as long as the input names stay the same. New inputs fall back to their default. **Removed inputs (most recently in V89: `pause_timer` and `notify_block_active`) should be deleted from existing automations so no leftovers remain in the configuration.**

> If the blueprint is imported via URL (*Re-import blueprint*), Home Assistant fetches the file directly from the source. A locally replaced file is overwritten in the process – so do not mix the two approaches. **Since V89 the `source_url` points to the correct path in the repository root, so re-importing works again.**

7. Complete input reference

Basics

Input	Default	Meaning
`rollo_entity`	–	The shutter to control (`cover`).
`himmelsrichtung`	–	Azimuth window of the facade. Determines when "the sun shines on this window". For unusual facades pick the option whose angular range fits best.
`brandalarm_entity`	`input_boolean.brandalarm_status`	While `on` , the blueprint performs no movements at all.
`hausmodus_entity`	–	The house mode helper.
`weather_entity`	–	Weather entity for plausibility checks.

Positions (0 % = closed, 100 % = open)

Input	Default	Meaning
`max_oeffnung`	94	Normal open position. Deliberately choose below 100 % if the curtain would otherwise hit its mechanical stop or if the position is not reported reliably at the very top.
`beschattung_pos_morgens`	60	Shading position before the switchover time.
`beschattung_pos_nachmittags`	60	Shading position after the switchover time.
`beschattung_wechsel_zeit`	13:00	Switchover point between the two shading positions. Rule of thumb: the moment the sun hits the facade head-on.
`lueftungs_pos`	15	Minimum height with a tilted window. Explicitly a lower bound, not a fixed position: if the shutter is higher, it stays where it is.
`durchgangs_pos`	80	Position with a fully opened window while shading or extreme-heat protection is active. Without active sun protection the shutter moves to `max_oeffnung` instead.
`hitze_extrem_pos`	10	Position during extreme heat.

Temperature thresholds

Input	Default	Meaning
`raumtemp_max_schwelle`	23.5 °C	Indoor temperature above which the heat alarm applies. With shading already active the blueprint internally uses the threshold minus 1 K as hysteresis.
`raumtemp_min_schwelle`	21.0 °C	Winter lock. Below this indoor temperature there is never any shading – the sun is allowed to heat. **Correctly implemented only from V89; up to V88 the lock did the opposite (shading when cold, raising blocked).**
`aussentemp_schwelle`	25.0 °C	Outdoor temperature above which shading is permitted. OR-combined with the indoor threshold.
`hitze_extrem_innen_ein`	25.0 °C	Indoor threshold at which extreme mode starts.
`hitze_extrem_innen_aus`	23.5 °C	Indoor threshold below which extreme mode ends.
`hitze_extrem_aussen_ein`	28.0 °C	Outdoor threshold for extreme mode.
`hitze_extrem_aussen_aus`	26.0 °C	Outdoor threshold below which extreme mode ends.

> **Keep a minimum gap:** there should be at least 2 K between `raumtemp_min_schwelle` and `raumtemp_max_schwelle` . With too narrow a window (e.g. 21 / 22 °C) the on and off points effectively coincide and the shutter oscillates. Proven: 20 °C / 22.5 °C.

Brightness and PV

Input	Default	Meaning
`pv_sensor`	empty	PV power. W or kW – detected automatically.
`pv_schwelle`	2000 W	Above this power it counts as "bright". At twice the threshold ("blazing") the weather check is skipped.
`lux_sensor`	empty	Illuminance sensor, ideally one per facade side.
`lux_schwelle`	25000	Above this value it counts as "bright". The default is often too high for outdoor sensors in the shade – for a sensor on the facade, practical values are more like 9,000–15,000 lx.

If both sensors fail or are left empty, the weather entity decides on its own (failover).

Times and presence

Input	Default	Meaning
`zeit_fruehestens`	06:30	Earliest opening on working days.
`sonnenaufgang_woche`	on	When active, working days additionally wait for sunrise.
`zeit_we_fruehestens`	08:00	Earliest opening on weekends, public holidays and in holiday mode.
`zeit_spaetestens`	09:30	Hard fallback: opening happens at this time even without presence.
`zeit_oeffnen_sperre`	11:00	End of the morning phase. After this a closed shutter counts as manually closed (see chapter 9).
`zeit_kontrolle`	22:00	Time of the evening check for open windows.
`sunset_offset`	30 min	Delay after sunset before night mode begins. Negative values close earlier. Since V87 the value acts on the night condition itself and therefore on every trigger (correctly implemented only from V88).
`praesenz_sensor`	empty	One or more presence detectors. If one reports `on` and the earliest time has been reached, the shutter opens.
`workday_sensor`	empty	Public-holiday detection. `off` = late opening time.

> **Precision of the sunset offset:** today's sunset is derived from `next_setting` minus one day. Around the equinoxes sunset shifts by up to roughly 6 minutes per day, so the calculated night start is off by that much – a little early in autumn, a little late in spring. In midsummer and midwinter the deviation approaches zero. Irrelevant for offsets of 30–60 minutes, worth knowing for very short values.

Notifications

Input	Default	Meaning
`notify_handy`	empty	The service name without prefix. With `handys` the blueprint calls `notify.handys`.

The inputs notify_block_active and pause_timer were removed in V89.

8. Recommended settings by facade orientation

Facade	Switchover time	Shading am / pm	Rationale
East	approx. 10:00	20–30 % / 30 %	Early, low sun causes strong glare. After midday the facade is in shade – shading then ends automatically because `sonne_auf_f` becomes false.
South	approx. 11:00–13:00	50–65 % / 30–65 %	High sun. Partial shading is often enough because the roof overhang helps.
West	approx. 15:00	60 % / 40 %	Low, aggressive afternoon sun. Close further in the afternoon than in the morning – the most common misconfiguration is exactly the reverse order.
North	any	–	Shading mostly unnecessary. The blueprint is still useful for ventilation, night closing and walk-through.

Commissioning procedure

1. Set up one shutter first and observe it for one or two sunny days.
2. Does it lower too rarely? → Lower the lux or PV threshold.
3. Does it lower too often or on cloudy days? → Raise the thresholds or let the weather check take effect.
4. Does it oscillate? → Increase the gap between `raumtemp_min` and `raumtemp_max`.
5. Only once one shutter runs cleanly, transfer the settings to the rest.

9. The manual override

The blueprint tries to detect whether a shutter has been moved by hand to a position it would never choose itself. If so, it holds back its daytime logic and respects the user's decision.

Detection works via the current position. Counted as "automatic" – with ±2 % tolerance:

- `max_oeffnung`, both shading positions, `lueftungs_pos`, `durchgangs_pos`, `hitze_extrem_pos`
- fully closed (≤ 5 %)

Any other position counts as a manual override. In addition, a closed shutter *after* the morning opening lock also counts as manually closed – whoever lowers the shutters again at 11 in the morning evidently wants to be left alone.

> **Important:** the override only holds back the daytime logic (shading, raising, extreme-heat protection). It is deliberately ignored for: fire alarm, walk-through/cross-ventilation and evening closing. Safety and window logic always take precedence.

> **Side effect when configuring:** if two positions happen to be very close together (e.g. ventilation 30 % and shading 30 %), the blueprint can no longer tell them apart. Choose positions at least 10 % apart.

10. Working with a fire-alarm automation

The blueprint does **not** open shutters on a fire alarm itself – it stays quiet. Opening them as an escape route is handled by a separate automation. This division of labour is deliberate: otherwise the two would overwrite each other.

Example of the corresponding automation (abridged):

```
alias: "Safety: FIRE ALARM"
mode: parallel
triggers:
  - trigger: state
    entity_id:
      - binary_sensor.rauchmelder_wohnzimmer
      - binary_sensor.rauchmelder_flur
      - binary_sensor.hitzemelder_kueche
    to: "on"
actions:
  # 1. Set the boolean -> the blueprint withdraws immediately
  - action: input_boolean.turn_on
    target:
      entity_id: input_boolean.brandalarm_status

  # 2. Critical push notification
  - action: notify.handys
    data:
      title: "FIRE ALARM!"
      message: "Triggered by: {{ trigger.to_state.name }}"
      data:
        priority: high
        channel: alarm_stream

  # 3. Open all shutters as an escape route
  - action: cover.set_cover_position
    target:
      entity_id:
        - cover.rollo_wohnzimmer
        - cover.rollo_schlafzimmer
    data:
      position: 100

  # 4. Escape route lighting
  - action: light.turn_on
    target:
      entity_id: light.flur
    data:
      brightness_pct: 100
```

The boolean is not reset on its own. As long as `input_boolean.brandalarm_status` is `on`, all shutter automations are dormant – even after a harmless false alarm from cooking fumes or dust. The automations still show as `on` and look perfectly healthy; they simply bail out into branch 0 on every run. The fault is practically invisible from the outside. The two building blocks below solve this.

10.1 Acknowledged reset via push notification

An automatic reset would be dangerous: after a real event you do not want the shutters closing again on their own. Hence the solution with human confirmation – two additional automations.

Automation A – offer the reset

Once all detectors have been quiet for 10 minutes and the boolean is still active, a push notification with action buttons arrives:

```yaml
alias: "Safety: FIRE ALARM offer reset"
mode: single
triggers:
  - trigger: state
    entity_id:
      - binary_sensor.rauchmelder_wohnzimmer
      - binary_sensor.rauchmelder_flur
      # ... all further detectors
    to: "off"
    for:
      minutes: 10
  - trigger: state
    entity_id: input_boolean.brandalarm_status
    to: "on"
    for:
      minutes: 10
conditions:
  - condition: state
    entity_id: input_boolean.brandalarm_status
    state: "on"
  - condition: state
    entity_id:
      - binary_sensor.rauchmelder_wohnzimmer
      - binary_sensor.rauchmelder_flur
      # ... all further detectors
    state: "off"
actions:
  - action: notify.handys
    data:
      title: "Fire alarm: detectors quiet again"
      message: >-
        All detectors have been clear for 10 minutes. The shutter control
        is still DORMANT. Only confirm once the situation is resolved.
      data:
        ttl: 0
        priority: high
        actions:
          - action: BRANDALARM_RESET
            title: "Reset alarm"
          - action: BRANDALARM_IGNORE
            title: "Not yet"
```

Automation B – reset confirmed

Reacts to the button press in the notification:

```yaml
alias: "Safety: FIRE ALARM reset confirmed"
mode: single
triggers:
  - trigger: event
    event_type: mobile_app_notification_action
    event_data:
      action: BRANDALARM_RESET
conditions:
  - condition: state
    entity_id: input_boolean.brandalarm_status
    state: "on"
actions:
  - action: input_boolean.turn_off
    target:
      entity_id: input_boolean.brandalarm_status
  - action: notify.handys
    data:
      title: "Fire alarm reset"
      message: >-
        The shutter control is active again. Normal operation within
        the next 5 minutes.
```

> If the button is never pressed, simply nothing happens – the shutters stay up. There is deliberately no timeout that takes the decision out of human hands. After the reset the shutters do not move all at once but take effect on the next regular run, i.e. within 5 minutes at most (`time_pattern`).

When creating these via the HA API, the validator may report that `BRANDALARM_RESET` is "not found in service registry". That is a false positive – it is a notification action ID, not a service name.

10.2 Dashboard card as a second safety net

In case the phone is not at hand, the boolean should also be reachable in the UI. A conditional card shows it only while the alarm is active and disappears by itself after the reset:

```
type: conditional
conditions:
  - condition: state
    entity: input_boolean.brandalarm_status
    state: "on"
card:
  type: tile
  entity: input_boolean.brandalarm_status
  name: FIRE ALARM — shutter control locked
  icon: mdi:fire-alert
  color: red
  features:
    - type: toggle
grid_options:
  columns: full
```

Sensible position: right at the top of the main dashboard, directly below the person badges – there it is impossible to miss in an emergency but does not get in the way during normal operation because it is invisible.

11. Troubleshooting and debugging

Reading traces

Settings → Automations → the affected automation → menu top right → *Traces*. There HA shows for every run which `choose` branch was taken and with which variable values. The *Changed Variables* tab is the most informative – it lists `sonne_auf_f`, `hitze_alarm`, `beschattung_aktiv`, `is_manual_override` and `aktuelle_pos` in plain text.

> If a run hits a template error, the trace shows an `error` field and no `execution` instead of a chosen branch. This is the fastest way to tell a faulty variable apart from a wrong branch: on a template error no branch is reached at all, and the shutter stops moving entirely – including the daytime logic.

Typical symptoms

Symptom	Cause and remedy
Shutter never shades	Check: indoor temperature above the winter lock? Sun within the azimuth window of the chosen orientation? Lux/PV threshold realistic? Does the weather entity report one of the recognised states?
Shutter shades constantly	Usually an outdoor threshold set too low in combination with the OR logic. Raise the outdoor threshold.
Shutter shades on cool days although the indoor temperature is below the winter lock	**Check the blueprint version. Up to V88 hitze_alarm was always true below the winter lock: branch 4b shaded in sunshine and branch 4c (raising) was blocked. Fixed in V89.**
Shutter oscillates	Temperature thresholds too close together, or two positions too close together (override detection cannot separate them cleanly).
Shutter does not rise in the morning	No presence detector triggered and `zeit_spaetestens` not yet reached. Or: `sonnenaufgang_woche` active and the sun has not risen yet in winter.
Shutter ignores the daytime logic	Most likely `is_manual_override`. Move the shutter to a known position once (e.g. fully open), then the automation takes over again.
Push notifications do not arrive	The notify service does not exist. Check under Developer tools → Actions whether `notify.<name>` is listed. After creating a notify group a restart is required.
Nothing moves at all	Check `input_boolean.brandalarm_status` – if it is `on`, the blueprint is dormant. The conditional dashboard card from chapter 10.2 makes this immediately visible. If the traces show a `TypeError` or another template error instead, a variable is faulty – see the note above.
Shutter closes right at sunset despite a configured offset	Check the blueprint version. Before V87 `sunset_offset` only affected the sunset trigger and was additionally defeated by `mode: restart`. From V87 the offset lives in the night condition.

Testing variables live

Developer tools → Template. This lets you check, for instance, whether the sun is currently on the facade:

```
Azimuth:   {{ state_attr('sun.sun', 'azimuth') }}
Elevation: {{ state_attr('sun.sun', 'elevation') }}
Position:  {{ state_attr('cover.rollo_wohnzimmer', 'current_position') }}
```

For the night start:

```
{% set offset = 15 %}
Sunset:      {{ (state_attr('sun.sun','next_setting') | as_datetime
                 - timedelta(days=1)) | as_local }}
Night start: {{ (state_attr('sun.sun','next_setting') | as_datetime
                 - timedelta(days=1)
                 + timedelta(minutes=offset)) | as_local }}
```

> `state_attr('sun.sun','next_setting')` returns a **string**, not a `datetime`. Without `| as_datetime` any arithmetic on it fails with a `TypeError`. This applies to your own templates just as much as to modifications of the blueprint.

> **Always write condition variables as a single expression, never as an `{% if %}` block.** A Jinja block does not return a boolean – it renders a multi-line string, and every non-empty string is truthy in Jinja. A block emitting the word `false` is therefore evaluated as *true*. Home Assistant only converts exact Python literals: `False`, capitalised. This is exactly what broke V87 (`ist_nacht` was true around the clock and the shutters closed at midday) **– and the same bug sat in the else branch of `hitze_alarm` up to V88, turning the winter lock into its opposite**. The correct form is a single expression using `and` / `or` that returns a real boolean. To verify in the template editor, print the raw value inside quotes – anything other than exactly `True` or `False` is a string.

12. Known limitations

- **Position-capable shutters only.** Pure open/close drives without `current_position` do not work.
- **No slat control.** Venetian blinds with slat angles are only controlled by height.
- **One shutter per automation.** That is intentional (individual parameters per window) but means a correspondingly large number of automations with many windows.
- **No shading calculation for neighbouring buildings or trees.** The azimuth range is a rough model. Anyone with a row of trees in front of

the west window has to adjust the angular range or the lux threshold accordingly.

- **No wind protection.** For awnings or external blinds at risk from wind, an additional higher-level lock would be needed.
- **The fire-alarm boolean is not reset by the blueprint.** That is intentional – the reset belongs in its own, acknowledged automation. See chapter 10.
- The `time_pattern` every 5 minutes means: a change to the conditions takes effect after 5 minutes at worst. This also applies to the start of night mode.
- Sunset is derived from `next_setting` minus one day and deviates by up to roughly 6 minutes around the equinoxes. See chapter 7.

13. Changelog

Version	Changes
V89	Fix: the **winter lock (`raumtemp_min_schwelle`) worked backwards**. Below the threshold, `hitze_alarm` emitted the bare word `false` from an `{% else %}` branch – the same bug class as V88: a non-empty string, therefore truthy. Below the winter lock `hitze_alarm` was always true, so branch 4b shaded whenever the sun hit the façade and PV/illuminance was high, and branch 4c (raise) was blocked via `not hitze_alarm` . On exactly the cool days when solar gain is wanted, the shutters went into shading. `hitze_alarm` is now a single boolean expression; behaviour above the winter lock (including the 1 °C hysteresis while shading) is unchanged. **Removed:** inputs `pause_timer` and `notify_block_active` plus the unused variables `v_timer` , `v_notify_block_raw` , `v_notify_block` and `pause_aktiv` . **Migration:** remove both inputs from existing automations; the notification-pause timer helper is no longer needed. **Corrected:** `source_url` points to the actual path in the repository root.
V88	Fix: V87 shipped a broken `ist_nacht` and **must not be used**. The condition was written as a `{% if %}` / `{% else %}` block that emitted the bare word `false` . A Jinja block does not return a boolean — it renders a multi-line *string* (`"\n false\n"`), and every non-empty string is truthy. Home Assistant only converts exact Python literals (`False` , capitalised), so lowercase `false` stayed a string and evaluated as **true**. `ist_nacht` was therefore true around the clock: branch 3 fired at any hour and drove the shutters to the ventilation or closed position in broad daylight, while branch 4 (`not ist_nacht`) never ran at all. Fixed by expressing the condition as a single `and` expression, which returns a real boolean: above the horizon → `False` , below → the time comparison. Lesson for the blueprint in general: any variable used as a condition must be one expression, never an `{% if %}` block.
V87	*(broken, superseded by V88)* Fix: the configured **sunset offset was effectively dead**. It sat as a `delay` inside branch 3 and only ran when `trigger.id == 'sunset_event'` . Every other trigger — presence sensor, window contact, PV, house mode, safety net — went straight through, and `ist_nacht` was already true the moment the sun dropped below the horizon. Shutters therefore closed within minutes of sunset even with 60 minutes configured. Second, deeper problem: even the sunset path never worked, because the automation runs in `mode: restart` — the pending delay was aborted by the next trigger, since V85 (`/15` → `/5`) after at most 5 minutes, and the restarted run satisfied `ist_nacht` and closed immediately. Solution: the offset now lives in the `ist_nacht` variable instead of a delay, so it applies to *every* trigger and is immune to `mode: restart` . The delay construct in branch 3 was removed without replacement. No new trigger is required — the existing `/5` safety net fires branch 3 as soon as `ist_nacht` flips, so the worst-case imprecision is 5 minutes. **All inputs are unchanged** (`sunset_offset` keeps its key, type, default and selector), so a re-import updates existing automations without any reconfiguration. Note on the implementation: `state_attr('sun.sun','next_setting')` returns a **string**, not a `datetime` , so the expression needs `| as_datetime` before the subtraction. Without it the template raises a `TypeError` , `ist_nacht` fails to render, and neither branch 3 (`ist_nacht`) nor branch 4 (`not ist_nacht`) is reached — the shutter stops moving entirely. Branch 4 was deliberately left on `not ist_nacht` : during the new dusk phase (sun down, offset still running) both `ist_hell` and `ist_nacht` are false, and 4c uses that window to raise shutters still parked in a shading position, while manually closed ones stay down via `is_manual_override` .
V86	Fix: branch 4c (raising) checked `not lueftung_aktiv` , which blocked raising for as long as a window was tilted – permanently, not just while shading was active. The shutter moved correctly into the shading or extreme-heat position via 4a/4b, but never came back up once the sun-position phase ended. Condition removed: throughout the blueprint the ventilation position is modelled as a **minimum height** – 1b only raises when the current position is below it, branch 3 closes only down to it in the evening, and 4a/4b do not check it either – not as a ceiling. `sperrschutz` remains in 4c, and since branch 4 only runs during the day, the night-time tilt logic in branch 3 is untouched.
V85	Fix: the PV trigger was a `state` trigger with `for: 2min` and practically never fired on a continuously changing power sensor, because the `for` window restarted on every value change – so shading never reacted to PV power directly, only on the periodic net. Now a `numeric_state` trigger (`above` / `below` `pv_schwelle` , each `for: 2min`), which debounces correctly. Change: safety net (`time_pattern`) tightened from `/15` to `/5` minutes so the fine reopening edge at `pv_schwelle − 500` / `lux_schwelle − 5000` is caught within 5 instead of 15 minutes. `mode: restart` and the position guards still prevent redundant travel.
V84	New: fully opened window raises the shutter completely when no sun-position mode is active (also at night, before opening time, from a manually closed state); only walkthrough position while shading is active. Fix: variable order – `sonne_auf_f` is now defined before `hitze_extrem` . New: workday sensor input for holiday detection. New: helper variables to disentangle the brightness logic.
V83	Extreme-heat protection with a separate position and hysteresis over indoor and outdoor temperature.
V82	Anti-flapping: debounced PV trigger, `mode: restart` , 15-minute safety net.

This documentation describes the blueprint "Master Roller Shutter Control V89". Use at your own risk – in particular the fire-alarm coupling should be tested end to end once before productive use (set the boolean by hand and check that all shutters react as expected).

Passages highlighted in red mark changes compared with the previous documentation revision (as of V88).